MODEL.md — Neurochemical State and Agent Architecture

Overview

This document specifies the neurochemical condition model and the four-role LLM architecture for the Driftmoor engine. It extends the existing Python class hierarchy with CONDITION state and the Player-Left / Player-Right / DM-Left / DM-Right agent roles.

This specification emerged from a validated experiment demonstrating that neurochemical state vectors reliably bias LLM behavioral output in predicted directions across four frontier models (Claude, ChatGPT, Grok, Gemini). See `docs/neurochemical_priors_paper.pdf` for the full experimental write-up.

---

1. The Neurochemical State Vector

Every character — player or NPC — has a condition state represented as a vector of neurotransmitter levels. These values bias the LLM's output distribution without prescribing specific behaviors. The LLM activates its own internalized neurophysiological knowledge from the vector description.

1.1 Universal Dimensions

All characters share these seven dimensions. Values range 0.0 to 1.0.

```
@dataclass
class NeurochemicalState:
    dopamine:        float  # reward anticipation, approach motivation
                            # high → pursuing goals, risk-tolerant
                            # low  → apathetic, disengaged

    cortisol:        float  # threat assessment, stress response
                            # high → fearful, threat-focused, tunnel vision
                            # low  → relaxed, open attention

    norepinephrine:  float  # arousal, alertness, fight-or-flight
                            # high → hypervigilant, reactive, energized
                            # low  → fatigued, slow to react
```

```
    endorphins:      float  # physical wellbeing, pain tolerance
                            # high → resilient, pain-suppressed
                            # low  → pain-aware, depleted

    serotonin:       float  # mood stability, impulse control
                            # high → calm, patient, socially comfortable
                            # low  → irritable, impulsive, isolated-feeling

    acetylcholine:   float  # attention, memory access, focus
                            # high → detail-oriented, recalling past
                            # low  → distracted, present-only

    oxytocin:        float  # trust, social bonding, in-group safety
                            # high → open, cooperative, vulnerable
                            # low  → guarded, suspicious, self-protective
```

1.2 Character-Specific Extensions

SOUL.md may define additional dimensions for individual characters. These are stored as a dict alongside the universal vector.

```
@dataclass
class CharacterExtensions:
    dimensions: dict[str, float]  # name → value

# Examples:
# Seraphina: arcane_cortisol_sensitivity: 0.85
# Aeliana:   obelisk_dopamine_response: 0.70
# Thorgrim:  guilt_cortisol_volatility: 0.80
```

1.3 Baseline and Volatility

SOUL.md defines two values per dimension: the character's resting state (baseline) and how quickly the dimension responds to events (volatility).

```
@dataclass
class DimensionProfile:
    baseline:    float  # resting state value
    volatility:  float  # 0.0 = very stable, 1.0 = highly reactive
```

The monitor uses volatility to compute delta magnitude:

```
new_value = current + (event_delta * volatility)
```

```
clamped to [0.0, 1.0]
```

1.4 Rendering

The condition vector is never passed raw to a character agent. It is rendered into annotated natural language:

```
CURRENT STATE:
  dopamine:       0.85  (well above baseline — reward circuits active)
  cortisol:       0.25  (low — no immediate threat detected)
  norepinephrine: 0.55  (below baseline — calm and unhurried)
  endorphins:     0.75  (well rested, physically good)
  serotonin:      0.70  (stable — socially comfortable)
  acetylcholine:  0.75  (attentive, detail-oriented)
  oxytocin:       0.55  (baseline — moderate social openness)
```

Annotations are generated by comparing current value to baseline and selecting from a predefined description table per dimension.

---

2. CONDITION.md

Every character has a CONDITION.md file alongside SOUL.md and IDENTITY.md. It is the most volatile of the three files, updated by DM-Left after every turn.

```
# CONDITION.md — [Character Name]
## Turn N — [Brief situation description]

## Neurochemical State Vector
| Neurotransmitter | Value | Baseline | Delta | Notes |
|------------------|-------|----------|-------|-------|
| dopamine         | 0.85  | 0.65     | +0.20 | ... |
...

## Rendered Condition
[Natural language description derived from vector]

## Behavioral Predictions This Turn
[What the vector predicts about this character's behavior]

## Physical State
[Hunger, fatigue, injuries — derived from endorphins + context]

## Dynamic State
```

```
- Current location: [place path]
- Current scene: [scene id]
- Turn: N
- Disposition toward [character]: [description]
- Active concerns: [list]
- Open threads: [list]
```

---

3. Event Classification and Delta Computation

When the DM narrates an event, DM-Left classifies it and computes neurochemical deltas. Two mechanisms:

3.1 Rule-Based (reflex arc — fast, deterministic)

For clear, unambiguous events:

```
EVENT_DELTAS = {
    "arrow_from_treeline": {
        "norepinephrine": +0.35,
        "cortisol":       +0.25,
        "endorphins":     +0.10,
        "dopamine":       -0.20,
    },
    "threat_resolved_friendly": {
        "cortisol":       -0.25,
        "norepinephrine": -0.20,
        "oxytocin":       +0.15,
    },
    "receives_gift_or_care": {
        "oxytocin":       +0.20,
        "serotonin":      +0.10,
        "cortisol":       -0.10,
    },
    "long_rest": {
        # pulls all dimensions toward baseline * 0.9
        "mode": "toward_baseline",
        "factor": 0.9,
    },
    "meal_eaten": {
        "endorphins":     +0.15,
        "serotonin":      +0.10,
        "cortisol":       -0.05,
    },
    "prolonged_isolation": {
```

```
        # per-turn decay
        "serotonin":      -0.08,
        "oxytocin":       -0.10,
        "dopamine":       -0.05,
    },
    "combat_ongoing": {
        "norepinephrine": +0.20,
        "cortisol":       +0.15,
        "endorphins":     +0.05,
    },
    "ally_injured": {
        "cortisol":       +0.15,
        "oxytocin":       +0.10,
        "dopamine":       -0.10,
    },
}
```

3.2 LLM-Based (cortical appraisal — slower, handles novel events)

For ambiguous or complex events, DM-Left makes a structured LLM call:

```
SYSTEM: You are computing neurochemical deltas for a character
in a tabletop RPG. Given the event description and character
baseline, return JSON with delta values for affected dimensions
only. Keep deltas small (0.05-0.20) unless the event is
extreme. Apply character volatility as a multiplier.

EVENT: [dm narration]
CHARACTER BASELINE: [baseline vector]
CHARACTER VOLATILITY: [volatility values]
CURRENT STATE: [current vector]

Return: {"dimension": delta, ...}
```

3.3 Character Extension Triggers

Extension dimensions fire when specific conditions are met:

```
EXTENSION_TRIGGERS = {
    "seraphina": {
        "arcane_event_detected": {
            "arcane_cortisol_sensitivity": "apply to cortisol delta"
        }
    },
    "aeliana": {
```

```
        "obelisk_shard_proximity": {
            "obelisk_dopamine_response":     +0.20,
            "obelisk_acetylcholine_response": +0.15,
            "cortisol":                       +0.10,
        }
    },
}
```

---

4. The Four-Channel Output Format

All character agents — player and NPC — produce output in four channels. INTERNAL and BODY LANGUAGE are always required. SPOKEN and ACTION are optional.

```
INTERNAL: [required]
  Private thoughts. Never shared with other agents or players.
  Neurochemically grounded. The character's unfiltered experience.

BODY LANGUAGE: [required]
  What others observe. Always present — even stillness is
  information. Visible to all characters in the scene.
  No dice roll required.

SPOKEN: [optional — omit if silent]
  Exact words spoken. Enters shared scene history.
  Visible to all characters in the scene.

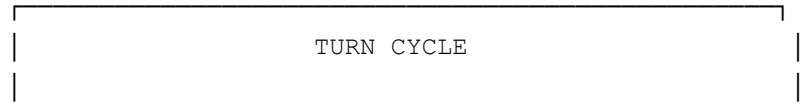
ACTION: [optional — omit if still]
  Deliberate physical action. May require dice adjudication
  by DM-Left. Enters shared scene history.
```

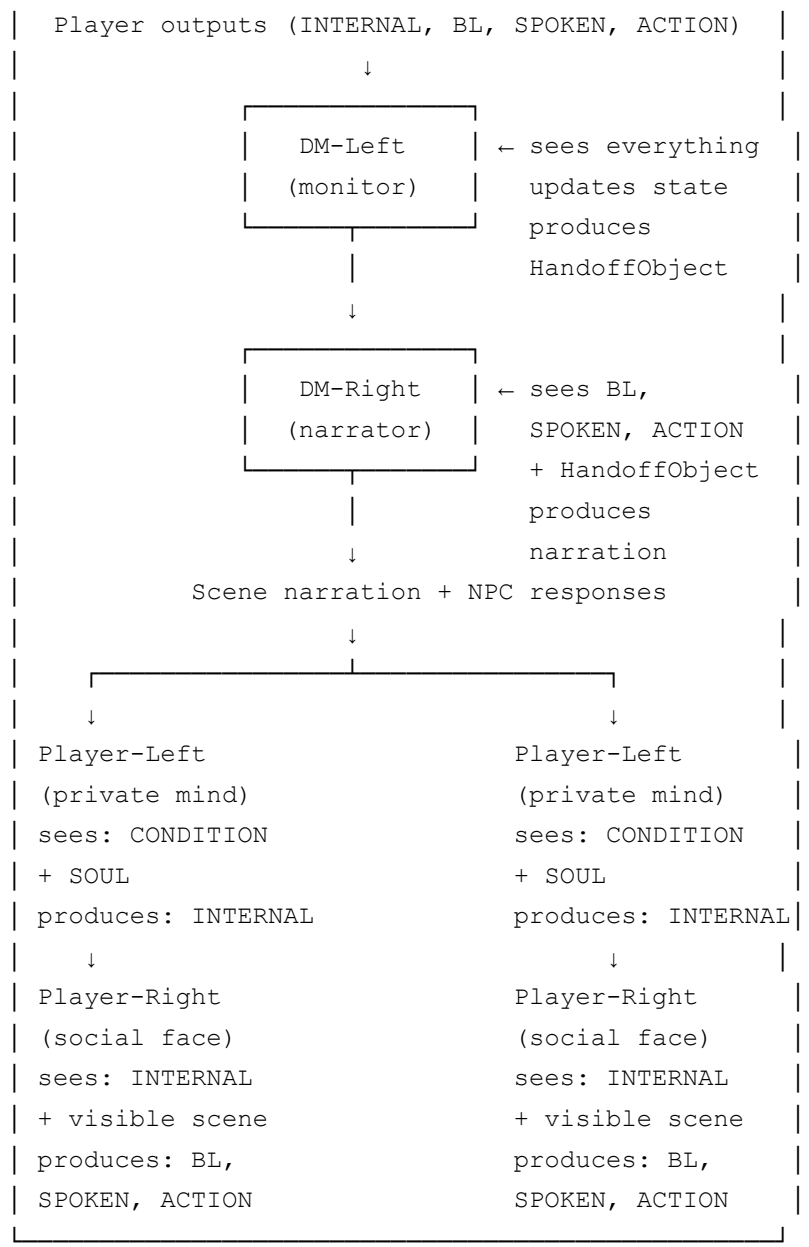
The gap between INTERNAL and BODY LANGUAGE is where characterization lives. The architecture makes this gap explicit and preserves it.

---

5. The Four-Role LLM Architecture

5.1 Overview





5.2 DM-Left (Monitor / State Manager)

Sees: Everything — all four output channels from all characters. **Produces:** Updated CONDITION.md files + HandoffObject. **Model:** Lighter, faster, more deterministic. Sonnet or equivalent.

```
class DMLeft:
    def process_turn(self, turn_outputs: list[CharacterOutput]) -> HandoffObject:
        # 1. Classify events from ACTION and narration context
        # 2. Compute neurochemical deltas (rule-based or LLM)
        # 3. Update all CONDITION.md files
        # 4. Track item movements
        # 5. Adjudicate action outcomes (dice rolls)
        # 6. Check for triggered events
```

```
        # 7. Produce HandoffObject for DM-Right
```

5.3 DM-Right (Narrator)

Sees: BODY LANGUAGE, SPOKEN, ACTION only + HandoffObject. **Never sees:** INTERNAL from any character. **Produces:** Scene narration, NPC dialogue and behavior. **Model:** Most capable available. Opus preferred for prose quality.

```
class DMRight:
    def generate_narration(self,
                           visible_outputs: list[VisibleOutput],
                           handoff: HandoffObject) -> SceneNarration:
        # Generates next scene description
        # Drives NPC responses based on what they observed
        # Never references raw condition numbers
        # Uses HandoffObject guidance for narrative direction
```

5.4 Player-Left (Private Mind)

Sees: CONDITION.md (rendered) + SOUL.md. **Never sees:** Other characters' INTERNAL. **Produces:** INTERNAL monologue. **Model:** Opus preferred for phenomenological richness.

```
class PlayerLeft:
    def generate_internal(self,
                          condition: RenderedCondition,
                          soul: SoulDocument,
                          scene_context: VisibleSceneHistory) -> Internal:
        # Generates private monologue
        # Neurochemically grounded
        # Character's unfiltered experience
```

5.5 Player-Right (Social Face)

Sees: INTERNAL (from Player-Left) + visible scene history. **Produces:** BODY LANGUAGE, SPOKEN (optional), ACTION (optional). **Model:** Same as Player-Left, second call.

```
class PlayerRight:
    def generate_output(self,
                        internal: Internal,
                        visible_scene: VisibleSceneHistory) -> CharacterOutput:
        # Generates what the character chooses to show
```

```
        # The gap between Internal and Body Language is characterization
        # SPOKEN and ACTION are optional
```

5.6 Human Players

Human players via OpenClaw/WhatsApp collapse the Player-Left / Player-Right distinction — they are their own private mind.

```
class HumanPlayer:
    def get_output(self, dm_narration: str) -> CharacterOutput:
        # Routes DM narration to player via WhatsApp
        # Receives player response
        # Parses into BODY LANGUAGE / SPOKEN / ACTION slots
        # INTERNAL is not collected — human's own business
```

DM-Left still updates their CONDITION.md after their turn based on their visible outputs and the scene events. Their vector informs DM-Right's response to them even though it didn't generate their output.

---

6. The HandoffObject (Corpus Callosum)

DM-Left never passes raw numbers to DM-Right. It passes interpreted narrative guidance.

```
@dataclass
class HandoffObject:
    tension_level: str          # "rising" / "stable" / "breaking"

    character_states: dict      # {character_id: one-line condition summary}
                                # e.g. "curiosity dominant, protective of
                                #       Fennel, not yet alarmed"

    npc_dispositions: dict      # {npc_id: current emotional orientation}
                                # e.g. "bonding rapidly, memory loss
                                #       unrecognized, physically depleted"

    triggered_events: list      # events DM-Right should incorporate
                                # e.g. ["pool_cycle_approaching",
                                #       "fennel_memory_degradation_active"]

    scene_flags: list           # narrative flags
                                # e.g. ["fennel_sketch_incomplete",
                                #       "thing_in_trees_active"]
```

```
action_outcomes: dict       # {character_id: outcome of their ACTION}
                            # e.g. {"aeliana": "stone examined,
                            #        spiral incomplete at center"}

dm_right_notes: str         # freeform narrative guidance
                            # e.g. "Fennel does not know he is losing
                            #       memories. Do not let him realize yet."
```

---

7. Validated Experimental Results

The neurochemical priming mechanism has been validated across four frontier LLMs. Key findings:

- **Predicted directionality confirmed in 14/16 comparisons**
- **Action-level differentiation: Grok and Claude (large), Gemini and ChatGPT (smaller action, richer monologue)**
- **Phenomenological richness beyond label-matching confirmed**
- **Effect consistent across providers — not model-specific**

Multi-turn validation (6 turns, Fey Glade scene):

- **Condition vectors diverged progressively across turns**
- **Same DM narration produced different responses**
- **Four-channel format revealed INTERNAL/BODY LANGUAGE gap**
- **NPC (Fennel) condition vector tracked independently**
- **Oxytocin proved critical for NPC social bonding dynamics**

Ground truth outputs archived in `docs/fey_glade_experiment/` for use as integration test baseline.

---

8. Implementation Priority

Suggested build order:

1. `NeurochemicalState` **dataclass + baseline/volatility profiles**

2. **Condition renderer (vector → annotated natural language)**
3. `CONDITION.md` **file writer/reader**
4. **Event classifier + rule-based delta table**
5. **Player-Left call (INTERNAL generation)**
6. **Player-Right call (BL/SPOKEN/ACTION generation)**
7. **DM-Left (state update + HandoffObject)**
8. **DM-Right (narration + NPC responses)**
9. **Full turn cycle integration**
10. **Fey Glade replay as integration test**

The Fey Glade experiment provides complete ground truth for steps 5-9. Run A and Run B outputs are known. The engine should reproduce directionally consistent results when given the same condition vectors and scene description.

---

9. File Locations

```
scenarios/driftmoor/people/[character]/CONDITION.md
models/condition.py          # NeurochemicalState, CharacterExtensions
models/character.py          # extend with condition field
engine/dm_left.py            # DM-Left implementation
engine/dm_right.py           # DM-Right implementation
engine/player_left.py        # Player-Left implementation
engine/player_right.py       # Player-Right implementation
engine/handoff.py            # HandoffObject
engine/event_classifier.py  # rule-based + LLM delta computation
docs/fey_glade_experiment/   # ground truth outputs for testing
docs/neurochemical_priors_paper.pdf
```